

Poisson ROM

From Training to Residual to Final Prediction

A complete numerical walkthrough of the separable-decoder architecture

Purpose of this document. The project dimensions in Section 2 are taken from the $N = 256$ Poisson quadrature-free configuration. The small matrices later in the document are deliberately constructed teaching numbers, not experimental measurements. They use the same algebra as the production implementation and are consistent with a real four-point discrete Poisson problem.

Contents

1	The problem and the model	2
2	The real project dimensions	2
3	Offline training	3
4	One-time Poisson precomputation	5
5	Online solve for a new source	7
6	Convergence and final prediction	8
7	What is fixed, and what changes?	9
8	Why the implementation looks slightly shorter	10
9	The entire method in twelve lines	10
	Plain-language glossary	11

1 The problem and the model

We want to solve the Poisson equation

$$-\Delta u = f, \quad u = 0 \text{ on the boundary.} \quad (1)$$

Here f is a new source field and u is the solution we want to predict.

The learned decoder has two separate neural networks:

$$u(\mathbf{x}; \mathbf{z}) = s b_c(\mathbf{x}) \mathbf{g}(\mathbf{x})^\top \mathbf{h}(\mathbf{z}). \quad (2)$$

The two networks have different jobs:

- $\mathbf{g}(\mathbf{x})$ produces R spatial-pattern values at coordinate $\mathbf{x}$. Here R means the number of learned spatial patterns.
- $\mathbf{h}(\mathbf{z})$ produces R coefficients, one coefficient for each of those R learned spatial patterns.
- $b_c(\mathbf{x})$ is zero on the boundary, so every decoded field satisfies the boundary condition automatically.

On all interior grid points at once, the decoder becomes

$$\mathbf{u}(\mathbf{z}) = \mathbf{G} \mathbf{h}(\mathbf{z}), \quad \mathbf{G} \in \mathbb{R}^{P \times R}, \quad (3)$$

Here P is the number of interior grid values in the solution vector, and R is the number of learned spatial patterns. Thus $\mathbf{G}$ has one row per grid value and one column per learned pattern. In the implementation, row i of $\mathbf{G}$ is

$$G_{i,:} = s b_c(\mathbf{x}_i) \mathbf{g}(\mathbf{x}_i)^\top, \quad (4)$$

so the fixed training-data scale s and the hard boundary mask are already inside the cached bank.

The new forcing f is *not* fed into $\mathbf{h}$. For a new problem, f enters the residual. A numerical solver changes $\mathbf{z}$ until the decoded field satisfies the projected Poisson equation.

Three sizes to remember throughout the document. P counts physical-grid values in $\mathbf{u}$; K counts the latent variables in $\mathbf{z}$ that the online solver changes; and R counts both the learned spatial patterns in $\mathbf{G}$ and their coefficients in $\mathbf{h}(\mathbf{z})$. The same reminder appears in every page footer.

2 The real project dimensions

For the $N = 256$ Poisson quadrature-free run, the full mesh has 256×256 points. Removing the outer boundary leaves

$$P = (256 - 2)^2 = 254^2 = 64,516 \quad (5)$$

interior points. In other words, P is the length of a full interior solution vector, not the number of latent variables or learned patterns.

The main dimensions are:

Object	Shape	Meaning
$\mathbf{z}$	$16 = K$	latent variables changed by the solver
$\mathbf{h}(\mathbf{z})$	$64 = R$	one coefficient per learned spatial pattern
$\mathbf{G}$	$64,516 \times 64 = P \times R$	grid values by learned patterns
$\Phi^\top$	$64 \times 64,516$	64 sine test functions
$\mathbf{B}$	64×64	projected spatial bank
$\mathbf{A}$	64×64	projected Poisson operator
$\mathbf{r}(\mathbf{z})$	64	residual seen by the latent solver
$\mathbf{J}_r$	64×16	residual Jacobian with respect to $\mathbf{z}$

The training configuration used 512 training solutions and 100,000 optimizer steps, with

$$K = 16, \quad R = 64, \quad M = 64. \quad (6)$$

That means 16 latent variables are adjusted online ($K = 16$), the decoder uses 64 learned spatial patterns and 64 matching coefficients ($R = 64$), and the residual retains 64 test equations ($M = 64$).

Notation warning. The blackboard-bold symbol $\mathbb{R}$ in an expression such as $\mathbb{R}^{16}$ means “vectors of real numbers.” It is not the model-size symbol R . For example, $\mathbb{R}^{16}$ below is a real-valued vector with 16 entries, while $R = 64$ counts learned patterns.

The spatial path begins with 64 learned frequency vectors, expanded into 64 sine values and 64 cosine values. Its architecture is

$$\mathbb{R}^2 \longrightarrow 128 \text{ Fourier features} \longrightarrow 128 \longrightarrow 128 \longrightarrow \mathbb{R}^{64}, \quad (7)$$

with SiLU activations after the two hidden dense layers and a linear final layer. Its output is multiplied by the fixed training-data RMS scale and the hard boundary mask when the bank is formed. The latent path is

$$\mathbb{R}^{16} \longrightarrow 128 \longrightarrow 128 \longrightarrow \mathbb{R}^{64}, \quad (8)$$

again with SiLU hidden activations and a linear final layer. A separate linear map from $\mathbb{R}^{16}$ to $\mathbb{R}^{64}$ is added as a skip connection. In this path, the 16 input entries are the $K = 16$ latent variables, and the 64 output entries are the $R = 64$ learned-pattern coefficients.

3 Offline training

3.1 Step 1: generate solution snapshots

The full-order Poisson solver produces training pairs

$$(\mathbf{f}_s, \mathbf{u}_s), \quad s = 1, \dots, 512. \quad (9)$$

Each $\mathbf{u}_s$ contains $P = 64,516$ interior solution values in the $N = 256$ run; here P again means the number of physical-grid values stored in one solution snapshot.

3.2 Step 2: assign a trainable code to every snapshot

Because this is an auto-decoder, there is no encoder. Every training snapshot gets its own trainable latent vector:

$$\mathbf{z}_s \in \mathbb{R}^{16}. \quad (10)$$

The 16 entries of this vector are the $K = 16$ latent variables assigned to that snapshot. The 512 latent codes and the weights of both neural networks are optimized together.

3.3 Step 3: reconstruct every training solution

At a training step, evaluate the spatial bank below. It has P rows, one for each interior grid value, and R columns, one for each learned spatial pattern:

$$\mathbf{G}_{\theta_g} = \begin{bmatrix} s b_c(\mathbf{x}_1) \mathbf{g}_{\theta_g}(\mathbf{x}_1)^\top \\ \vdots \\ s b_c(\mathbf{x}_P) \mathbf{g}_{\theta_g}(\mathbf{x}_P)^\top \end{bmatrix} \in \mathbb{R}^{P \times R} \quad (11)$$

The coefficient matrix below has 512 rows, one for each training snapshot, and R columns, one coefficient for each learned spatial pattern:

$$\mathbf{H}_{\theta_h} = \begin{bmatrix} \mathbf{h}_{\theta_h}(\mathbf{z}_1)^\top \\ \vdots \\ \mathbf{h}_{\theta_h}(\mathbf{z}_{512})^\top \end{bmatrix} \in \mathbb{R}^{512 \times R}. \quad (12)$$

All training reconstructions are produced together by the following matrix. It has 512 rows, one reconstructed snapshot per row, and P columns, one for each interior grid value:

$$\hat{\mathbf{U}} = \mathbf{H}_{\theta_h} \mathbf{G}_{\theta_g}^\top \in \mathbb{R}^{512 \times P}. \quad (13)$$

The main loss is the global relative mean-square reconstruction error,

$$\mathcal{L}_{\text{recon}} = \frac{\|\hat{\mathbf{U}} - \mathbf{U}\|_F^2}{\|\mathbf{U}\|_F^2}, \quad (14)$$

plus a small feature-Gram regularizer that discourages badly conditioned or redundant columns in $\mathbf{G}$.

Adam updates

$$\theta_g, \quad \theta_h, \quad \mathbf{z}_1, \dots, \mathbf{z}_{512}. \quad (15)$$

3.4 A tiny training calculation

For the worked example, use only four interior points, two latent variables, and three spatial features:

$$P = 4, \quad K = 2, \quad R = 3. \quad (16)$$

Thus $P = 4$ means four physical-grid values in each solution, $K = 2$ means two latent variables for the solver to adjust, and $R = 3$ means three learned spatial patterns and three matching coefficients.

Suppose one training target is

$$\mathbf{u}_{\text{train}} = [1 \quad 0 \quad 1 \quad 2]^\top. \quad (17)$$

An early prediction might be

$$\hat{\mathbf{u}} = [0.3 \quad -0.1 \quad 0.2 \quad 0.4]^\top. \quad (18)$$

Its pointwise error and plain mean-square error are

$$\hat{\mathbf{u}} - \mathbf{u}_{\text{train}} = [-0.7 \quad -0.1 \quad -0.8 \quad -1.6]^\top, \quad (19)$$

$$\text{MSE} = \frac{0.49 + 0.01 + 0.64 + 2.56}{4} = 0.925. \quad (20)$$

This plain MSE is only a small arithmetic illustration; the production loss uses the relative normalization written above.

After many updates, suppose training has produced the bank

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 2 & 1 & -1 \end{bmatrix}. \quad (21)$$

For the training snapshot above, suppose the trained head gives

$$\mathbf{h}(\mathbf{z}_{\text{train}}) = [1 \ 0 \ 0]^\top. \quad (22)$$

Then

$$\mathbf{G}\mathbf{h}(\mathbf{z}_{\text{train}}) = [1 \ 0 \ 1 \ 2]^\top, \quad (23)$$

so this teaching snapshot is reconstructed exactly.

3.5 Step 4: freeze the learned model

After training, the spatial network, Fourier frequencies, latent network, and linear skip are frozen. The training codes are no longer optimized. They may still be useful for constructing an initial guess for a new solve.

4 One-time Poisson precomputation

4.1 Step 5: evaluate the frozen spatial bank

Evaluate the frozen spatial network at every interior point, multiply by the fixed output scale and hard boundary mask, and build a matrix with P rows (one per interior grid value) and R columns (one per learned spatial pattern):

$$\mathbf{G} \in \mathbb{R}^{P \times R}. \quad (24)$$

This is the large spatial calculation. It is not repeated during each latent solver iteration.

4.2 Step 6: construct the sine test functions

Let

$$\Phi = [\phi_1 \ \cdots \ \phi_M] \in \mathbb{R}^{P \times M} \quad (25)$$

contain the selected discrete sine modes. The P rows place every test function on all P interior grid values; the M columns are the M retained test functions. They are eigenvectors of the discrete negative Laplacian $\mathbf{L}$:

$$\mathbf{L}\phi_m = \lambda_m\phi_m. \quad (26)$$

For a general fixed linear operator, the projected matrix would be built as

$$\mathbf{A} = \Phi^\top \mathbf{L} \mathbf{G}. \quad (27)$$

For Poisson sine modes, symmetry and the eigenvector identity give the shortcut

$$\mathbf{A} = \mathbf{\Lambda} \mathbf{B}, \quad \mathbf{B} = \Phi^\top \mathbf{G}, \quad (28)$$

where

$$\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_M). \quad (29)$$

4.3 Step 7: compute every small matrix in the toy example

Take a 4×4 full grid on $[0, 1]^2$. It has a 2×2 interior, so $P = 4$ physical interior-grid values, and its grid spacing is $\Delta x = 1/3$. For readable arithmetic, manually select two test modes, $M = 2$. Their values are

$$\Phi^\top = \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & -0.5 & 0.5 & -0.5 \end{bmatrix}. \quad (30)$$

These are actual normalized two-dimensional discrete sine modes on this tiny grid. The production mode selector retains complete eigenvalue shells. On this unusually small grid, requesting two modes would therefore retain the second mode's tied partner as a third mode. The teaching example intentionally omits that tied partner to keep every displayed matrix two rows tall; each displayed vector remains a valid sine eigenmode. Their negative-Laplacian eigenvalues are

$$\lambda_1 = 18, \quad \lambda_2 = 36, \quad \Lambda = \begin{bmatrix} 18 & 0 \\ 0 & 36 \end{bmatrix}. \quad (31)$$

Projecting the bank in Equation (21) gives

$$\mathbf{B} = \Phi^\top \mathbf{G} \quad (32)$$

$$= \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & -0.5 & 0.5 & -0.5 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 2 & 1 & -1 \end{bmatrix} \quad (33)$$

$$= \begin{bmatrix} 2 & 1.5 & 0.5 \\ 0 & -0.5 & 0.5 \end{bmatrix}. \quad (34)$$

For example, the upper-left entry is

$$B_{11} = 0.5(1) + 0.5(0) + 0.5(1) + 0.5(2) = 2. \quad (35)$$

The projected Poisson matrix is

$$\mathbf{A} = \Lambda \mathbf{B} \quad (36)$$

$$= \begin{bmatrix} 18 & 0 \\ 0 & 36 \end{bmatrix} \begin{bmatrix} 2 & 1.5 & 0.5 \\ 0 & -0.5 & 0.5 \end{bmatrix} \quad (37)$$

$$= \begin{bmatrix} 36 & 27 & 9 \\ 0 & -18 & 18 \end{bmatrix}. \quad (38)$$

For the project choice $\alpha = 1$, the residual weighting is the inverse eigenvalue scaling

$$\mathbf{W} = \Lambda^{-1} = \begin{bmatrix} 1/18 & 0 \\ 0 & 1/36 \end{bmatrix}. \quad (39)$$

At this point, $\mathbf{G}$, Φ , $\mathbf{B}$, $\mathbf{A}$, Λ , and $\mathbf{W}$ are mathematically fixed. The optimized code does not need to materialize all of them as separate dense matrices: it stores $\mathbf{B}$, the eigenvalue vector, and the already scaled forcing.

5 Online solve for a new source

5.1 Step 8: receive and project the new forcing

Suppose the new forcing on the four interior points is

$$\mathbf{f} = [54 \quad 9 \quad 9 \quad 18]^\top. \quad (40)$$

Project it onto the two test functions once:

$$\mathbf{f}_M = \Phi^\top \mathbf{f} \quad (41)$$

$$= \begin{bmatrix} 0.5(54) + 0.5(9) + 0.5(9) + 0.5(18) \\ 0.5(54) - 0.5(9) + 0.5(9) - 0.5(18) \end{bmatrix} \quad (42)$$

$$= \begin{bmatrix} 45 \\ 18 \end{bmatrix}. \quad (43)$$

This source projection may touch the full grid, but it is performed only once per new source, not once per latent iteration. If the forcing has a known affine expansion, the projections of its fixed components can also be precomputed.

Notation warning. In this document, $\mathbf{f}_M$ means the raw projection $\Phi^\top \mathbf{f}$. The implementation variable named `f_m` instead stores the scaled vector $\Lambda^{-1} \Phi^\top \mathbf{f}$.

5.2 Step 9: evaluate one candidate latent state

Suppose the latent solver proposes

$$\mathbf{z} = [0.4 \quad -0.7]^\top \quad (44)$$

and the frozen latent MLP returns

$$\mathbf{c} = \mathbf{h}(\mathbf{z}) = [2 \quad -1 \quad 0.5]^\top. \quad (45)$$

The projected left side of the PDE is

$$\mathbf{A}\mathbf{c} = \begin{bmatrix} 36 & 27 & 9 \\ 0 & -18 & 18 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \\ 0.5 \end{bmatrix} \quad (46)$$

$$= \begin{bmatrix} 36(2) + 27(-1) + 9(0.5) \\ 0(2) - 18(-1) + 18(0.5) \end{bmatrix} \quad (47)$$

$$= \begin{bmatrix} 49.5 \\ 27 \end{bmatrix}. \quad (48)$$

The raw projected residual is

$$\mathbf{A}\mathbf{h}(\mathbf{z}) - \mathbf{f}_M = \begin{bmatrix} 49.5 \\ 27 \end{bmatrix} - \begin{bmatrix} 45 \\ 18 \end{bmatrix} = \begin{bmatrix} 4.5 \\ 9 \end{bmatrix}. \quad (49)$$

After applying the weights,

$$\mathbf{r}(\mathbf{z}) = \mathbf{W}(\mathbf{A}\mathbf{h}(\mathbf{z}) - \mathbf{f}_M) \quad (50)$$

$$= \begin{bmatrix} 4.5/18 \\ 9/36 \end{bmatrix} = \begin{bmatrix} 0.25 \\ 0.25 \end{bmatrix}. \quad (51)$$

The candidate is therefore not yet a solution.

5.3 Step 10: compute a latent update

Automatic differentiation is needed only through the small head. Suppose its local Jacobian is

$$\mathbf{J}_h = \frac{\partial \mathbf{h}}{\partial \mathbf{z}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0.5 & -0.5 \end{bmatrix} \in \mathbb{R}^{3 \times 2}. \quad (52)$$

Its three rows correspond to the $R = 3$ learned-pattern coefficients, while its two columns correspond to the $K = 2$ latent variables being adjusted. Then the residual Jacobian is

$$\mathbf{J}_r = \mathbf{W} \mathbf{A} \mathbf{J}_h \quad (53)$$

$$= \begin{bmatrix} 2.25 & 1.25 \\ 0.25 & -0.75 \end{bmatrix}. \quad (54)$$

For illustration, an undamped local Newton step solves

$$\mathbf{J}_r \delta \mathbf{z} = -\mathbf{r}, \quad (55)$$

which here gives

$$\delta \mathbf{z} = \begin{bmatrix} -0.25 \\ 0.25 \end{bmatrix}. \quad (56)$$

The production method uses Levenberg–Marquardt damping and a trust-region limit, so it may take a smaller step. It repeats

$$\mathbf{z} \longrightarrow \mathbf{h}(\mathbf{z}) \longrightarrow \mathbf{r}(\mathbf{z}) \longrightarrow \delta \mathbf{z} \quad (57)$$

until the residual or progress stopping condition is satisfied.

No spatial network, full-grid Laplacian, or $\Phi^\top \mathbf{G}$ product is evaluated inside this loop.

6 Convergence and final prediction

6.1 Step 11: reach a zero-residual coefficient vector

Suppose the final latent state $\mathbf{z}_\star$ produces

$$\mathbf{h}(\mathbf{z}_\star) = [1 \quad 0 \quad 1]^\top. \quad (58)$$

Then

$$\mathbf{A} \mathbf{h}(\mathbf{z}_\star) = \begin{bmatrix} 36(1) + 27(0) + 9(1) \\ 0(1) - 18(0) + 18(1) \end{bmatrix} \quad (59)$$

$$= \begin{bmatrix} 45 \\ 18 \end{bmatrix} = \mathbf{f}_M. \quad (60)$$

Therefore

$$\mathbf{r}(\mathbf{z}_\star) = \mathbf{0}. \quad (61)$$

6.2 Step 12: decode the full field once

The final interior prediction is

$$\mathbf{u}_\star = \mathbf{G}\mathbf{h}(\mathbf{z}_\star) \quad (62)$$

$$= \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \\ 2 & 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \quad (63)$$

$$= \begin{bmatrix} 2 \\ 1 \\ 1 \\ 1 \end{bmatrix}. \quad (64)$$

Restoring the zero boundary gives the 4×4 field

$$\mathbf{U}_\star = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 2 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}. \quad (65)$$

For completeness, the full four-point negative-Laplacian matrix is

$$\mathbf{L} = 9 \begin{bmatrix} 4 & -1 & -1 & 0 \\ -1 & 4 & 0 & -1 \\ -1 & 0 & 4 & -1 \\ 0 & -1 & -1 & 4 \end{bmatrix}. \quad (66)$$

Direct multiplication verifies that this deliberately constructed toy prediction satisfies the complete grid equation, not only the two retained tests:

$$\mathbf{L}\mathbf{u}_\star = \begin{bmatrix} 54 \\ 9 \\ 9 \\ 18 \end{bmatrix} = \mathbf{f}. \quad (67)$$

In a general reduced solve, a small retained test set enforces only the selected weak equations; this full equality occurs here because the teaching example was constructed to make it checkable end to end.

7 What is fixed, and what changes?

Fixed after training/precomputation	Changes for a new source or solver iterate
weights of $\mathbf{g}$ and $\mathbf{h}$	the source field $\mathbf{f}$
spatial bank $\mathbf{G}$	its projected vector $\mathbf{f}_M$ changes per source
test functions Φ	current latent vector $\mathbf{z}$
eigenvalues Λ	coefficients $\mathbf{h}(\mathbf{z})$
projected matrices $\mathbf{B}$ and $\mathbf{A}$	residual and residual Jacobian
weighting $\mathbf{W}$	final decoded field

The matrices must be rebuilt if the learned spatial bank, mesh, boundary conditions, test space, or PDE operator changes. A new right-hand side on the same problem requires a new $\mathbf{f}_M$, but not a new $\mathbf{A}$.

8 Why the implementation looks slightly shorter

The slide form is

$$\mathbf{r}(\mathbf{z}) = \mathbf{W} [\mathbf{\Lambda} \mathbf{B} \mathbf{h}(\mathbf{z}) - \mathbf{f}_M], \quad \mathbf{W} = \mathbf{\Lambda}^{-1}. \quad (68)$$

Since $\mathbf{W} \mathbf{\Lambda} = \mathbf{I}$, this is exactly

$$\mathbf{r}(\mathbf{z}) = \mathbf{B} \mathbf{h}(\mathbf{z}) - \mathbf{\Lambda}^{-1} \mathbf{f}_M. \quad (69)$$

The quadrature-free code uses this second form: it materializes $\mathbf{B} = \mathbf{\Phi}^\top \mathbf{G}$, not the conceptual dense matrix $\mathbf{A}$, and stores the forcing after inverse-eigenvalue scaling. The apparent disappearance of $\mathbf{W}$ and $\mathbf{\Lambda}$ is only an algebraic fold; it does not change the residual.

9 The entire method in twelve lines

1. Generate full-order Poisson training solutions.
2. Give each training solution a trainable latent code.
3. Train $\mathbf{g}$, $\mathbf{h}$, and the training codes to reconstruct the solutions.
4. Freeze both neural networks.
5. Evaluate $\mathbf{g}$ on the grid to construct $\mathbf{G}$.
6. Construct the sine test functions $\mathbf{\Phi}$ and eigenvalues $\mathbf{\Lambda}$.
7. Precompute $\mathbf{B} = \mathbf{\Phi}^\top \mathbf{G}$ and, conceptually, $\mathbf{A} = \mathbf{\Lambda} \mathbf{B}$.
8. For a new source, compute $\mathbf{f}_M = \mathbf{\Phi}^\top \mathbf{f}$ once.
9. Choose an initial latent vector $\mathbf{z}$.
10. Repeatedly evaluate the small head and residual, then update $\mathbf{z}$.
11. Stop at $\mathbf{z}_\star$ when the residual is sufficiently small.
12. Decode once: $\mathbf{u}_\star = \mathbf{G} \mathbf{h}(\mathbf{z}_\star)$.

The central computational saving: the full grid is used to build the spatial bank, to project a new source, and to return the final field. It is not revisited during every latent-solver iteration. Each iteration uses only the small latent MLP and precomputed dense matrices.

Plain-language glossary

K — **latent variables**. The number of entries in $\mathbf{z}$ that the online solver adjusts.

R — **learned spatial patterns**. The number of columns in $\mathbf{G}$ and, equivalently, the number of coefficients returned by $\mathbf{h}(\mathbf{z})$.

M — **test equations**. The number of weak test equations retained by the ROM.

P — **physical-grid values**. The number of interior grid values stored in one solution vector $\mathbf{u}$.

$\mathbf{g}(\mathbf{x})$ — **spatial neural network**. Returns the raw learned-pattern values at coordinate $\mathbf{x}$.

$\mathbf{h}(\mathbf{z})$ — **latent neural network**. Returns the coefficients that combine the learned spatial patterns.

$\mathbf{G}$ — **frozen spatial bank**. Contains the learned patterns evaluated at every interior grid point, including the fixed output scale and hard boundary mask.

Φ — **test-function matrix**. Its columns are the selected sine test functions.

$\mathbf{B}$ — **projected bank**. This is $\Phi^\top \mathbf{G}$.

Λ — **eigenvalue matrix**. The diagonal matrix of Laplacian eigenvalues for the sine test modes.

$\mathbf{A}$ — **projected Poisson operator**. This equals $\Lambda \mathbf{B}$ in this setting.

$\mathbf{W}$ — **residual weighting**. The fixed inverse-eigenvalue scaling.

$\mathbf{f}_M$ — **projected forcing**. The new forcing projected onto the retained test functions.

$\mathbf{r}(\mathbf{z})$ — **residual**. The weighted mismatch between the decoded field's projected Poisson response and the projected forcing.

Auto-decoder. A decoder trained with one free latent code per training snapshot, without a learned encoder.

Offline. Training and precomputation performed before solving new source instances.

Online. The latent solve and final field reconstruction for a new source.